Can Language Models Repair Neural-Network Weights from Numerical Values? A Controlled Negative Result

Anonymous authors
Paper under double-blind review

Abstract

We ask whether a large language model, given a small trained network's weights as numbers together with labelled examples of its errors, can edit those weights so that the fix generalises. In a controlled setting (one six-state regular language, one 16-unit recurrent architecture with 321 parameters, a fixed numeric edit budget, and no ability to execute the network) two LLM editors each received the frozen outcome no_repair: their mean gain over no edit was at most the prespecified threshold $f = 0.0224$ on four of five seeds, while the gradient-access projected-Adam comparator exceeded f on all five. In an earlier unconstrained version of the task, two LLM constructors reached perfect accuracy; inspection showed they had reconstructed the target language from the labels and compiled a new solution into the weights. We report both results under their frozen decision rules, with their corrections and limits, and argue that when labels may identify the target function, a weight-editing study should report the edit route and include a labels-only reconstruction baseline.

1 Setup

The target is a binary string language accepted by a hidden minimal six-state deterministic finite automaton (DFA). The network is a tanh recurrent network with 16 hidden units: 32 input weights, 256 recurrent weights, 16 hidden biases, 16 readout weights and one readout bias, 321 parameters in all. Evaluation uses 500 hidden held-out strings of length 1 to 12 and 200 longer strings of length 13 to 24, never shown to any arm. The prespecified practical-effect threshold is $f = 0.0224$, one binomial standard error of an accuracy at 0.5 with $n = 500$; it is a threshold, not a significance bound for paired gains. The label-bearing arms receive 32 new oracle-labelled strings per round.

Experiment 1 (construction) gave the editor a free hand: it writes a Python function that returns the weights, run in a sandbox, with the 32 initial labels and accumulated reported failures in its prompt. Three arms share one starting point written by the editor from the 32 initial labels: adversarial queries chosen by a second model call, random queries, and self-refinement with no new labels. One seed, two editor models (Claude Opus 5.5; OpenAI gpt-6-astra as a replication on the same seed).

Experiment 2 (constrained numeric editing) started from a network trained by gradient descent on 32 initial labels chosen so that they do not identify the automaton, and allowed the editor only a list of numeric edits: at most 24 coordinate changes per round, at most 48 distinct coordinates over three rounds, and no coordinate farther than 1.0 from its trained value. This budget was set to exclude the measured one-hot compilation route: in preflight, compiled and trained weights differed on all 321 coordinates, with max-norm displacement 19.98 to 39.26; it does not exclude every sparse or symbolic route. Three arms from the same trained start and the same frozen random label batches: the editor (two independent realizations per seed), projected dense Adam under the identical budget, and no edit. Projected Adam has gradient access the editor lacks; it is a fixed comparator, not an optimal

Table 1: Experiment 2, per seed: trained starting accuracy (D0) and held-out gain after round 3 over no edit. Opus is the primary strict-format run; astra ran under the relaxed format; Adam is projected dense Adam under the identical 24/48/1.0 budget. $f = 0.0224$.

seed	D0	Opus 5.5	gpt-6-astra	projected Adam
1	0.692	−0.004	−0.007	+0.028
15	0.738	+0.066	+0.018	+0.118
20	0.750	+0.008	+0.026	+0.034
21	0.728	0.000	−0.037	+0.052
24	0.722	+0.007	−0.108	+0.044

sparse optimiser and not a lower bound on feasible repair. The frozen outcome no_repair means the editor's mean gain over no edit is at most f on at least 4 of 5 seeds. Seeds were chosen offline for a trained starting accuracy of at least 0.60 and for projected Adam being able to improve them, so that the comparator is informative; this makes the stratum favourable to the comparator by construction, and the result is conditional on it.

Primary runs used rules frozen before the first model call. One secondary rerun was specified after execution and is reported as descriptive.

2 Results

Experiment 1. The Opus constructors embedded 32 to 37 labelled examples literally, searched for a smallest consistent automaton, and compiled it into saturated recurrent weights. Both query arms reached held-out accuracy 1.000 in exactly the round in which the labels first identified the automaton uniquely, as checked offline by a SAT solver with an ambiguity re-solve. Self-refinement never improved and ended at 0.500. The astra replication produced the same mechanism (32 to 40 embedded examples) and the same identification-locked jump to 1.000 in both query arms. Before identification an inference contribution could not be excluded: on the same 32 labels, astra's label-free arm reached 0.924 where the solver's single pick scored 0.888. The experiment measured how many labels identify the automaton, a quantity the solver computes without a language model, and the design was stopped under its frozen rule.

Experiment 2. Table 1 gives, per seed, the trained starting accuracy and the held-out gain after round 3 for each editor (mean of two realizations) and for projected Adam. Both editors received no_repair: gain at most f on 4 of 5 seeds, Opus exceeding f only on seed 15 and astra only on seed 20. Projected Adam exceeded f on every seed. In the secondary relaxed-format comparison, which puts both editors under the same output protocol, the two produced different edit patterns: Opus ended with 1 to 12 touched coordinates of the 48 allowed and had 13 of 30 readout-only rounds, whereas astra ended with 17 to 36 touched coordinates and produced both gains (+0.034, +0.036) and losses (−0.114, −0.130) across seeds.

Follow-up calibration. In a separate frozen calibration on disjoint development seeds 4, 6, 25, and 28, two sparse black-box searches with forward-pass-only access, under the same 24/48/1.0 constraints, failed a prespecified 3-of-4 repair criterion at 100, 1,000 and 10,000 full-labelled-set objective evaluations per round; the best repaired 2 of 4 seeds at 10,000. A separately reported dense evolution strategy, excluded from the criterion because it ignores the sparsity constraint, also repaired 2 of 4 at 10,000. The projected-Adam reference exceeded f on only 1 of these 4 seeds.

3 Discussion

Limitations and corrections. The result shows that in this setting, with weights presented as numbers and no execution access, two current models received no_repair under a budget that the fixed projected-Adam comparator satisfies, and the separate unconstrained

construction experiment showed that both editors reconstructed the target language from labels and compiled a new network. It does not show that language models cannot repair weights in general: one task, one architecture, five seeds selected for gradient feasibility, two models, one budget, a fixed comparator with gradient access, and no execution access. Three corrections are part of the record. A preflight label-accumulation mismatch changed seed eligibility; we added seed 1 under the identical configuration and made the production-order set primary, with the same verdict on the original set. In the original five-seed run, 28 of 30 first attempts from Opus violated the required JSON-only format and the other two hit the output limit; a relaxed-parser rerun accepted 29 of 30 and reproduced the verdict with the same small edits, but was specified after execution and is descriptive. The 39 rejected proposals, scored post hoc on the reused 500-string held-out set without multiplicity correction, exceeded f only on seed 15.

Related work. Treating weights as data, including generating them, is an active direction (Ha et al., 2016; Han et al., 2026); direct editing of a trained model's parameters from examples is studied for factual associations in transformers (Meng et al., 2022; Mitchell et al., 2022); and language models have been used as optimisers over text-described objectives (Yang et al., 2023). Our task is the classical problem of identifying a regular language from labelled strings (Gold, 1978; Angluin, 1987), which is what the unconstrained editors solved.

Implication. In this task, both constructor editors reconstructed the target language from the labels and compiled a new solution, rather than repairing the supplied network by the intended mechanism; the resulting accuracy is real, but it answers a different question. When labels may identify the target function, weight-editing studies should report what the model changed, how much, and by what route, and should include a labels-only reconstruction baseline.

Reproducibility. Code, frozen rules, run records, every model attempt with its parse and budget status, and the analysis scripts will be released after terms and privacy review.

References

Dana Angluin. Learning regular sets from queries and counterexamples. Information and Computation, 75(2):87–106, 1987. doi: 10.1016/0890-5401(87)90052-6.

E. Mark Gold. Complexity of automaton identification from given data. Information and Control, 37(3):302–320, 1978. doi: 10.1016/S0019-9958(78)90562-4.

David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks. arXiv preprint arXiv:1609.09106, 2016.

Xiaolong Han, Zehong Wang, Bo Zhao, Binchi Zhang, Jundong Li, Damian Borth, Rose Yu, Haggai Maron, Yanfang Ye, Lu Yin, and Ferrante Neri. A survey of weight space learning: Understanding, representation, and generation. arXiv preprint arXiv:2603.10090, 2026.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In Advances in Neural Information Processing Systems, 2022. doi: 10.52202/068431-1262.

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast model editing at scale. In International Conference on Learning Representations, 2022.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. arXiv preprint arXiv:2309.03409, 2023.